Stage 11E-GR1 Common Budgeted Grid Planning

Scientific logical-grid decisions, deterministic ladders, field reuse, and backend-second selection

mdstats

2026-07-27

Contents

1 Purpose

Stage 11E-GR1 owns the scientific decision that converts a requested Cartesian resolution into one periodic logical grid or an exactly nested logical-grid ladder. The decision is made under a scientific density resource policy and is completed before dense, local-sparse, or any later execution backend is chosen.

GR1 builds on the Stage 11E-GR0 cell-metric and diagnostic records. It does not construct density values, alter the fixed kernel, select a visual bandwidth, extract a mesh, or admit a browser scene.

2 Normative separation

The order is mandatory:

```
physical target interval
    -> logical periodic grid
    -> scientific budget verdict
    -> frozen logical-grid signature
    -> backend feasibility estimates
    -> backend selection
```

A backend cannot repair an infeasible physical target by silently changing the grid or kernel. Dense and local-sparse candidates for the same field therefore carry the same logical-grid and fixed-kernel signatures.

Scientific resource limits are accepted through `ScientificDensityResourcePolicy` or an explicit logical-voxel limit used by focused compatibility tests. Plotly, mesh, browser, trace, scene, HTML, and rendering policies are not valid planner inputs.

3 Target and finest-feasible planning

`plan_finest_feasible_density_grid` accepts:

```
display cell
target Cartesian interval
coarsest admissible Cartesian interval
scientific resource policy or explicit logical-voxel limit
optional immutable metadata
```

The target shape is

$$N_i = \max\left(4, \left\lceil \frac{\|\mathbf{a}_i\|}{\Delta_{\text{target}}} - 10^{-12} \right\rceil\right).$$

When the target shape fits, the selected shape is exactly the target shape and the status is `target_reached`.

When the target does not fit but the coarsest admissible shape does, the planner uses the established deterministic 80-step interval bisection to return the finest automatic shape within the logical-voxel limit. The status is `budget_limited`, the reason contains `unresolved_due_to_resolution_budget`, and the result is not promoted as a scientifically converged resolution.

The legacy adaptive-broadening sentinel `target_interval=0` means "find the finest grid admitted by the scientific limit." It is retained only as a compatibility numerical request. The signed target geometry records that the original target was unbounded rather than pretending that zero is a physical Cartesian interval.

If the coarsest admissible shape itself exceeds the limit, planning raises `DensityNumericalResourceError`; no partial grid is returned.

4 `DensityLogicalGridPlan`

The immutable record contains:

```
target_geometry
selected_geometry
coarsest_geometry
max_logical_voxels
scientific_resource_signature
status
reason_codes
logical_grid_signature
metadata
signature
```

Each geometry retains both the requested interval, when finite, and the realized Cartesian intervals. The selected logical grid is signed independently of any backend.

The valid statuses are:

- `target_reached`;
- `budget_limited`;
- `level_limited`, reserved for a bounded nested ladder.

A `budget_limited` plan is valid diagnostic evidence but is not a scientific resolution certificate.

5 Deterministic nested-grid ladder

`plan_deterministic_density_grid_ladder` starts from the grid shape resolved at the coarsest interval. For integer refinement factor $r \geq 2$, level $k+1$ is constructed as

$$\mathbf{N}_{k+1} = r\mathbf{N}_k.$$

The grids are therefore exactly nested in every periodic lattice direction. For the default factor two, every parent voxel contains exactly $2^3 = 8$ child voxels.

The ladder stops when the longest realized Cartesian interval satisfies the requested finest interval, when the next exact nested level exceeds the scientific voxel limit, or when `max_levels` is reached. These outcomes are respectively:

```
target_reached
budget_limited / unresolved_due_to_resolution_budget
level_limited / unresolved_due_to_ladder_depth
```

A budget-limited record retains the finest feasible level and the first requested infeasible level. It does not relabel the feasible level as the requested finest grid.

6 `DensityNestedGridLadder`

The immutable ladder contains:

```
levels
requested_finest_geometry
coarsest_interval
finest_interval
refinement_factor
max_levels
max_logical_voxels
scientific_resource_signature
status
reason_codes
metadata
signature
```

Construction rejects non-nested shapes, mixed display cells, levels above the scientific limit, inconsistent statuses, and unsupported serialized schemas.

7 Logical-grid identity

`density_logical_grid_signature` hashes the cell, logical shape, realized intervals, and voxel volume. It deliberately excludes storage backend and rendering policy. Two backend candidates can describe the same scientific field only when this signature is identical.

8 Identical-field reuse

`DensityFieldReuseKey` binds:

```
field kind
source signature
sample-selection signature
weight signature
fixed-kernel signature
logical-grid signature
normalization signature
```

Its cache key is backend-independent. Operational metadata such as whether a cache hit came from dense or local-sparse storage does not alter the numerical field identity.

`require_identical_density_field_reuse` fails closed when any source, sample, weight, kernel, logical-grid, normalization, or field-kind identity differs. No partial signature match authorizes reuse.

9 Backend-second planning

`DensityBackendCandidatePlan` records one backend estimate bound to a frozen logical-grid signature and fixed-kernel signature. It contains feasibility, storage, workspace, work, and explicit infeasibility reasons.

`select_density_backend_after_grid` accepts one `DensityLogicalGridPlan` and candidate records. It rejects candidates that change the grid or kernel. An explicit backend request must be feasible. Automatic selection uses a stable ordering by estimated scientific work, workspace, storage, and backend name.

The resulting `DensityBackendSelectionPlan` retains the logical-grid-plan signature, logical-grid signature, fixed-kernel signature, all candidate records, the selected backend, and the deterministic selection reason.

This stage does not replace plotting's established dense/local-sparse policy. Stage 11E-GR2 adapts atomic and framework plotting to the common GR0/GR1 layer while preserving visual and browser behavior.

10 Serialization and signatures

Every GR1 persistent record is immutable, JSON-serializable, and signed with a canonical SHA-256 payload. Deserialization recomputes signatures and rejects payload tampering. Derived cache keys and logical-grid identities are also verified when serialized.

11 Failure behavior

GR1 fails closed for:

- invalid or singular cells;
- negative target intervals or nonpositive coarsest intervals;
- refinement factors below two;
- nonpositive level or resource limits;
- a coarsest grid that exceeds the scientific limit;
- mixed cells inside one plan or ladder;
- non-nested ladder shapes;
- backend candidates that change the frozen grid or kernel;
- field reuse with differing numerical identities;
- infeasible forced backends;
- signature or cache-key tampering;
- rendering-policy objects supplied in place of scientific resource policy.

Budget and level exhaustion are signed unresolved outcomes when at least one valid logical grid exists. They are not silently promoted.

12 Compatibility boundary

The plotting-private `_finest_budgeted_grid_shape` name remains available as a compatibility adapter. It delegates to GR1 and translates analysis numerical input/resource failures into `GraphComplexityError`. The selected shape and budget-limited Boolean remain regression-compatible, including the historical zero-target sentinel.

No other atomic/framework plotting ownership moves in GR1.

13 Acceptance tests

Focused tests cover:

- target-reached planning in a strongly triclinic cell;
- exact requested and realized interval retention;
- budget-limited finest-feasible parity with the plotting oracle;
- zero-target adaptive compatibility;
- coarsest-grid resource failure;
- deterministic exact factor-two nesting;
- budget-limited and level-limited ladders;
- signed JSON replay and tamper rejection;
- backend-independent field cache keys;
- fail-closed source/weight/kernel/grid mismatch;
- physical-resolution-first/backend-second enforcement;
- infeasible forced backend handling;
- absence of rendering and graph-policy imports.

The affected density diagnostics, broadening, atomic-density, and backend selection regressions must remain unchanged.

14 Implementation status

Implemented in `0.20.24a0`. Runtime owner is `mdstats.analysis.density.planning`; the plotting compatibility boundary remains in `mdstats.plotting.atomic_density`. Architecture revision 55 retains GR1 as implemented and records GR2 plotting adaptation as complete. Stage 11E-GR3 is the next implementation stage.